Beliefs representation and management

Autonomous Software Agents

A.A. 2025-2026

Prof. Paolo Giorgini

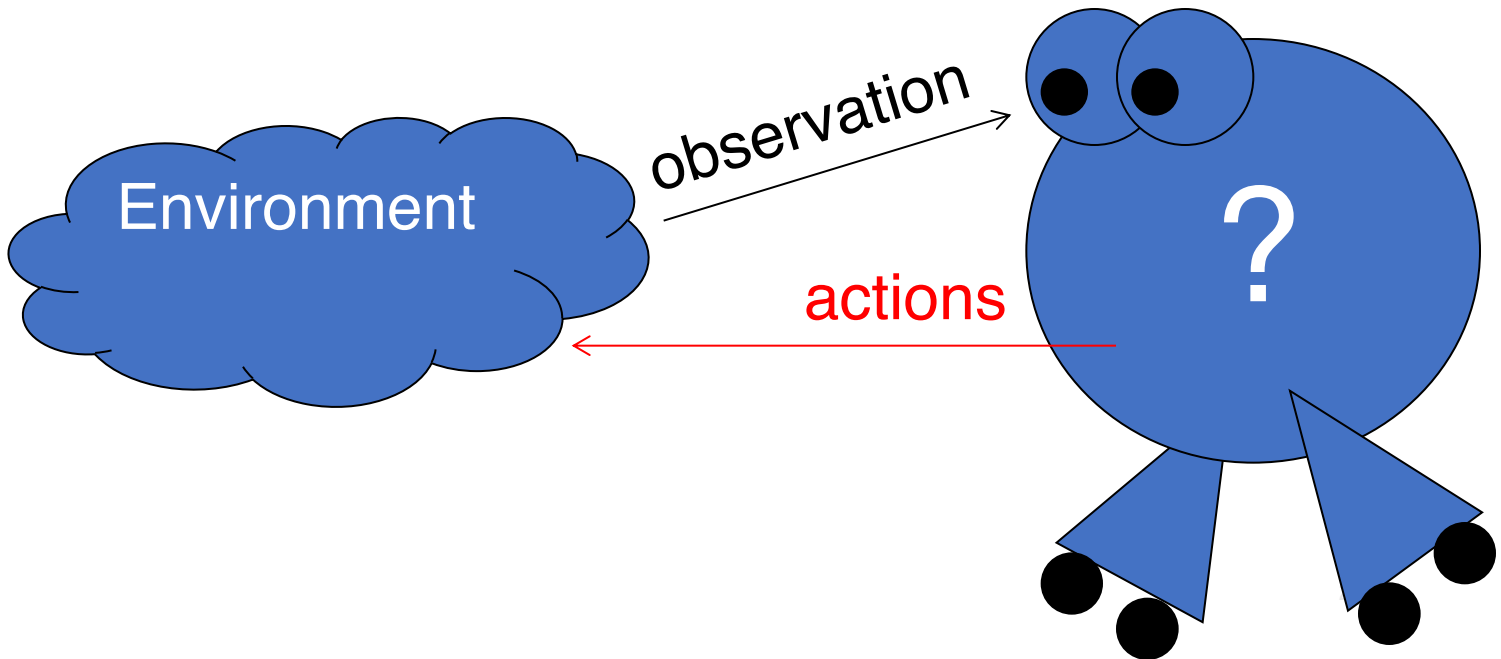


UNIVERSITY OF TRENTO - Italy

Department of Information and Communication Technology

Belief Revision

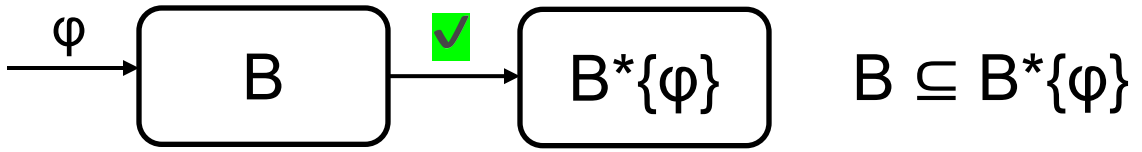
- An agent revise beliefs in response to new information
  - But which ways of revising beliefs are “OK” and which are not?
- A belief revision theory is meant to provide a general answer, with a sense of “OK” that it specifies



© P. Giorgini

Belief Revision Theory

- A belief revision theory is, roughly, a theory saying which ways of belief revision are OK and which are not
- **Preservation**. If the information that agent A receives at time t is *compatible* with the set of the beliefs that A has right before t, then, right after t, agent A retains all of her beliefs in response to that information
- If ϕ is compatible with B, then B is a subset of $B * \phi$, where:
  - B is the set of one's beliefs right before the receipt of new information,
  - ϕ is the new information one receives,
  - $B * \phi$ is the set of one's new beliefs in response to new information ϕ



$B \subseteq B^*\{\varphi\}$

Methodological questions

- How are the beliefs in the database ***represented***?
  - Most databases work with elements like ***facts*** and ***rules*** as primitive forms of representing information. The code used to represent the beliefs may be more or less closely related to standard logical formalism. A mechanism for belief revision is sensitive to the formalism chosen to represent the beliefs
- What is the relation between the elements explicitly represented in the database and the beliefs that may be ***derived*** from these elements?
  - It depends on the *application area*. The nature of the relation between explicit and implicit beliefs is of crucial importance for how the belief revision process is attacked
- How are the choices concerning what to ***retract*** *made*?
  - Eg: An agent can decide adopting the minimal change principle or on the base of the importance of the beliefs. In CS, Integrity constraints is a common way of handling the problem

Updates vs. Revisions

- Updating vs Revision - Katsuno and Mendelzon (1992)
  (1) new belief about a static world – **revision**
    - The information you receive corrects your knowledge about a world that has not changed
    - You need to modify your beliefs to align them with a reality that was already the case, but which you previously misunderstood
  (2) new belief about changes brought about by some agent – **updating**
    - Your beliefs were correct, but now the world has changed
    - The information you receive describes a change in the world
    - You need to modify your beliefs to reflect a new state of the world

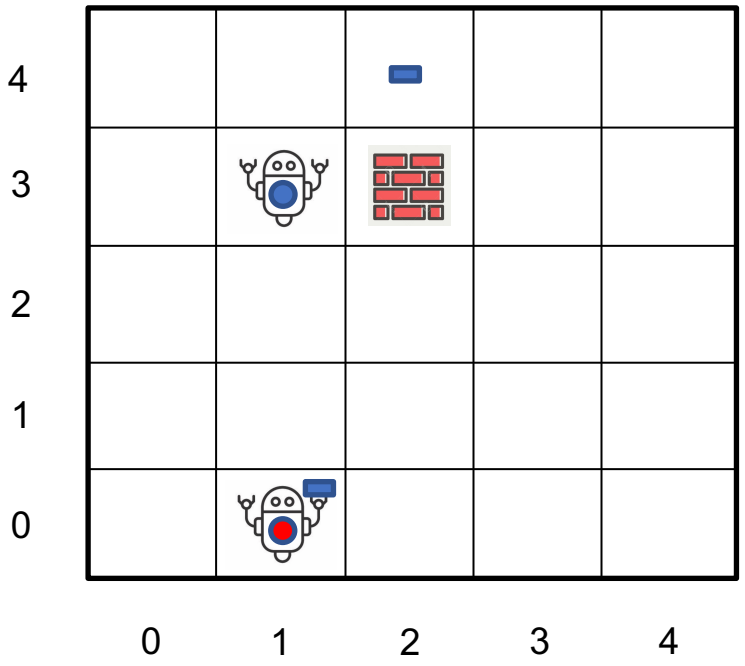
From sensors' data to Belief

- An agent acquires data from its sensors
  - E.g., temperature from the thermostat or its position from GPS

Time	Temperature	X	Y
09:00:00	8	3	3
10:00:00	9	3	4
11:00:00	11	3	5
12:00:00	13	4	6

  - Data can be stored as they are acquired
    - Acquisition time tells us actual values and it draws data evolution

Data completeness and correctness



Agent “Ag_1” (or ”Ag_2”)

Positions

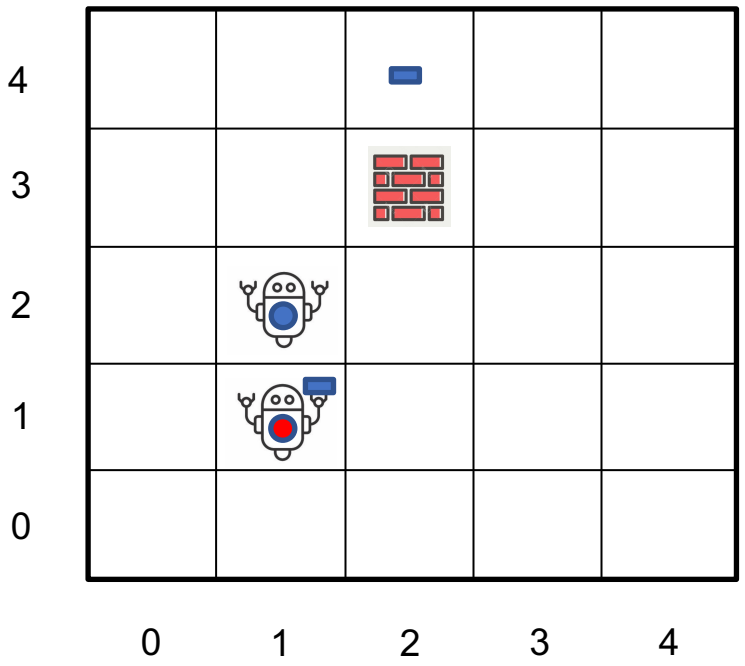
Time	Object	X	Y
1	Ag_1	1	0
1	Ag_2	1	3
1	Obst_1	2	3
1	Pack_1	2	4

Carry

Time	Agent	Pack
1	Ag_1	Pack_2

```
B = { In(Ag_1,1,0), In(Ag_2,1,3), In(Obst_1,2,3),
      In(Pack_1,2,4), carry(Ag_1,Pack_2) }
```

Data updating



Positions

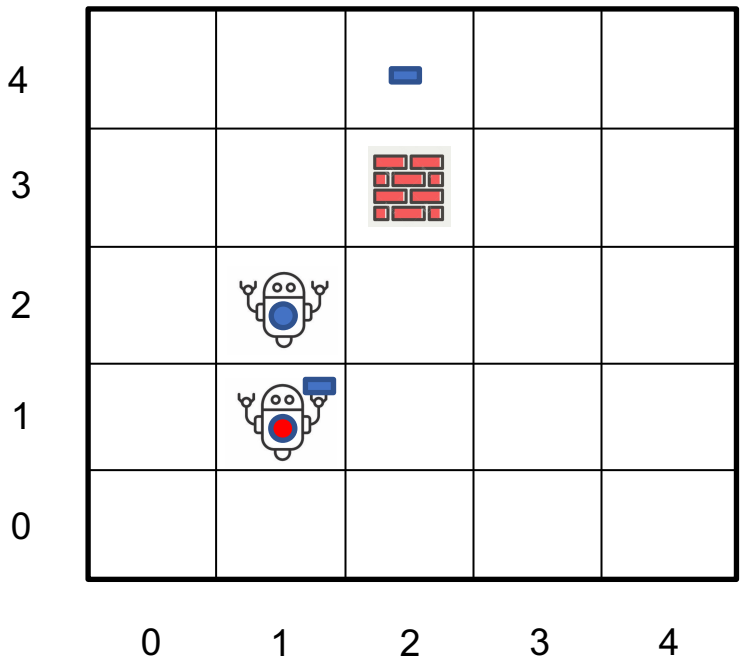
Time	Object	X	Y
~~1~~	~~Ag_1~~	~~1~~	~~0~~
~~1~~	~~Ag_2~~	~~1~~	~~3~~
~~1~~	~~Obst_1~~	~~2~~	~~3~~
~~1~~	~~Pack_1~~	~~2~~	~~4~~
2	Ag_1	1	1
2	Ag_2	1	2
2	Obst_1	2	3
2	Pack_1	2	4

Carry

Time	Agent	Pack
~~1~~	~~Ag_1~~	~~Pack_2~~
2	Ag_1	Pack_2

```
B = { In(Ag_1,1,1), In(Ag_2,1,2), In(Obst_1,2,3),
      In(Pack_1,2,4), carry(Ag_1,Pack_2) }
```

Derived Beliefs



Positions

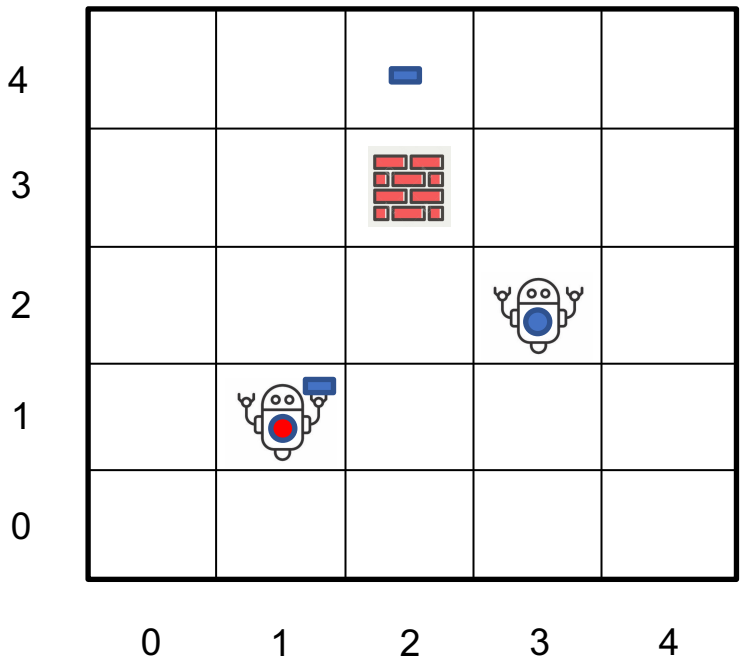
Time	Object	X	Y	Move
1	Ag_1	1	0	No
1	Ag_2	1	3	No
1	Obst_1	2	3	No
1	Pack_1	2	4	No
2	Ag_1	1	1	UP
2	Ag_2	1	2	DOWN
2	Obst_1	2	3	No
2	Pack_1	2	4	No

Carry

Time	Agent	Pack
1	Ag_1	Pack_2
2	Ag_1	Pack_2

```
B = { In(Ag_1,1,1), In(Ag_2,1,2), In(Obst_1,2,3),
      In(Pack_1,2,4), carry(Ag_1,Pack_2),
      move(Ag_1,UP),move(Ag_2,DOWN) }
```

Managing Inconsistencies



Positions

Time	Object	X	Y	Move
1	Ag_1	1	0	No
1	Ag_2	1	3	No
1	Obst_1	2	3	No
1	Pack_1	2	4	No
2	Ag_1	1	1	UP
2	Ag_2	1	2	DOWN
2	Obst_1	2	3	No
2	Pack_1	2	4	No
3	Ag_1	1	1	No
3	Ag_2	3	2	RIGHT
3	Obst_1	2	3	No
3	Pack_1	2	4	No

Carry

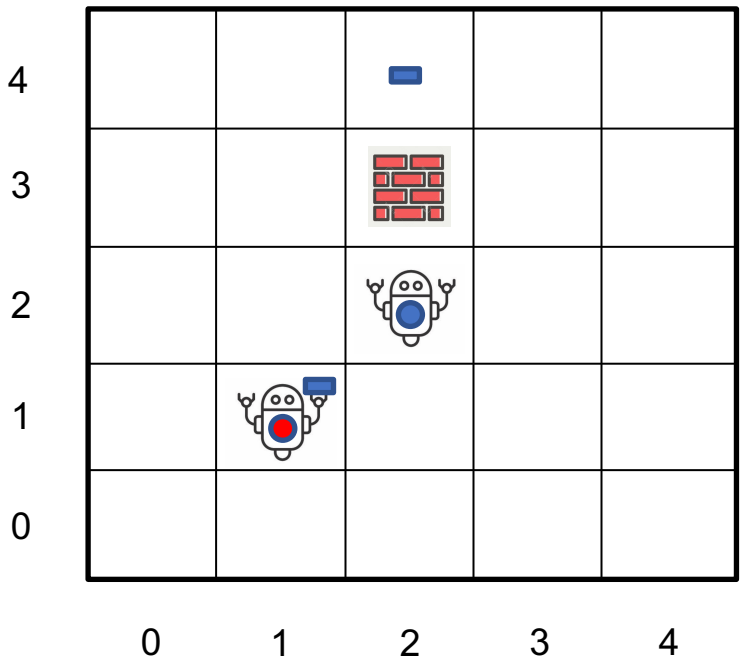
Time	Agent	Pack
1	Ag_1	Pack_2
2	Ag_1	Pack_2

Robot cannot move of two tiles in one step

Inconsistencies

- Inconsistencies can arise for several reasons
  - Sensors can send wrong data
  - Predefined rules are not anymore valid (evolution of the environment)
    - Now the robot can move of two tiles
  - Data are provided by other agents
    - They might lie or they could have wrong beliefs
- How much is it critical to solve the inconsistency?
  - Can we wait a little bit?
- How to solve them ?
  - Many different ways can be applied
    - `t=2 :In(Ag_2,1,2), move(Ag_2,DOWN)`
    - `t=3 :In(Ag_2,3,2), move(Ag_2,RIGHT)` ---→ `t=3 :In(Ag_2,2,2), move(Ag_2,RIGHT)`
- Policies/strategies to solve and manage inconsistencies should be part of the design

Managing Inconsistencies



Positions

Time	Object	X	Y	Move
1	Ag_1	1	0	No
1	Ag_2	1	3	No
1	Obst_1	2	3	No
1	Pack_1	2	4	No
2	Ag_1	1	1	UP
2	Ag_2	1	2	DOWN
2	Obst_1	2	3	No
2	Pack_1	2	4	No
3	Ag_1	1	1	No
3	Ag_2	2	2	RIGHT
3	Obst_1	2	3	No
3	Pack_1	2	4	No

Carry

Time	Agent	Pack
1	Ag_1	Pack_2
2	Ag_1	Pack_2

Solving the inconsistency
(most likely position for Ag_2?)

Managing inconsistencies

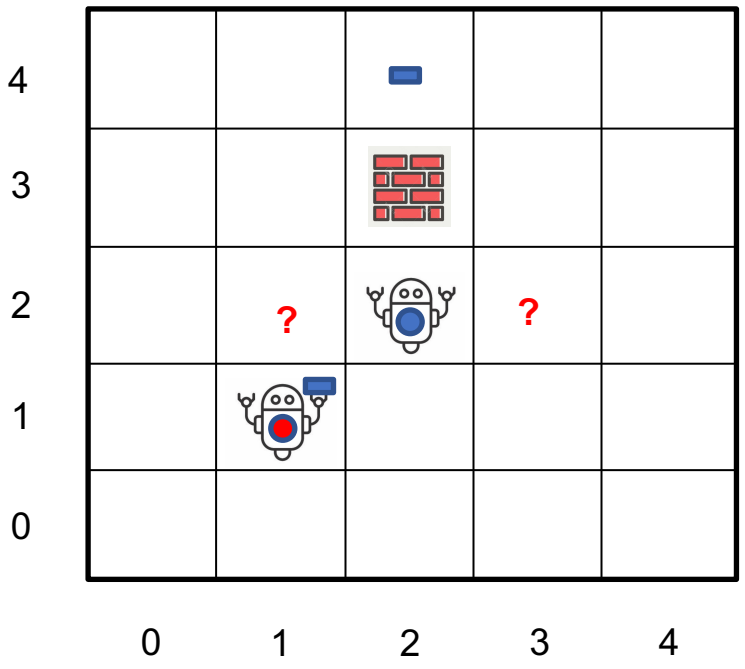
```
t=1: In(Ag_2,1,3)
t=2: In(Ag_2,1,2), move(Ag_2,DOWN)
t=3: In(Ag_2,3,2), move(Ag_2,RIGHT)
```

Possibile consistent sets:

```
t=1: In(Ag_2,1,3)
t=2: In(Ag_2,1,2), move(Ag_2,DOWN)
```

```
t=3: In(Ag_2,3,2), move(Ag_2,RIGHT)
```

Another example



t=1: In(Ag_2,**2**,**2**)
t=2: In(Ag_2,1,2), move(Ag_2,LEFT)
t=3: In(Ag_2,3,2), move(Ag_2,RIGHT)

Possibile consistent sets:

S_1

t=1: In(Ag_2,2,2)
t=2: In(Ag_2,1,2), move(Ag_2,LEFT)

S_2

t=1: In(Ag_2,2,2)
t=3: In(Ag_2,3,2), move(Ag_2,RIGHT)

More on the example

S_1

```
t=1: In(Ag_2,2,2)
t=2: In(Ag_2,1,2), move(Ag_2,LEFT)
```

what about t=3 ?

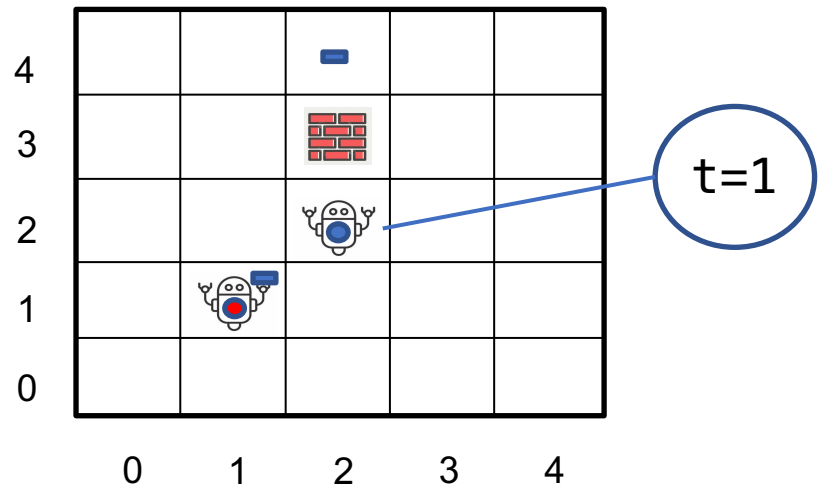
t=3: In(Ag_2,1,2) v In(Ag_2,0,2) v
In(Ag_2,1,3) v In(Ag_2,2,2)

S_2

```
t=1: In(Ag_2,2,2)
t=3: In(Ag_2,3,2), move(Ag_2,RIGHT)
```

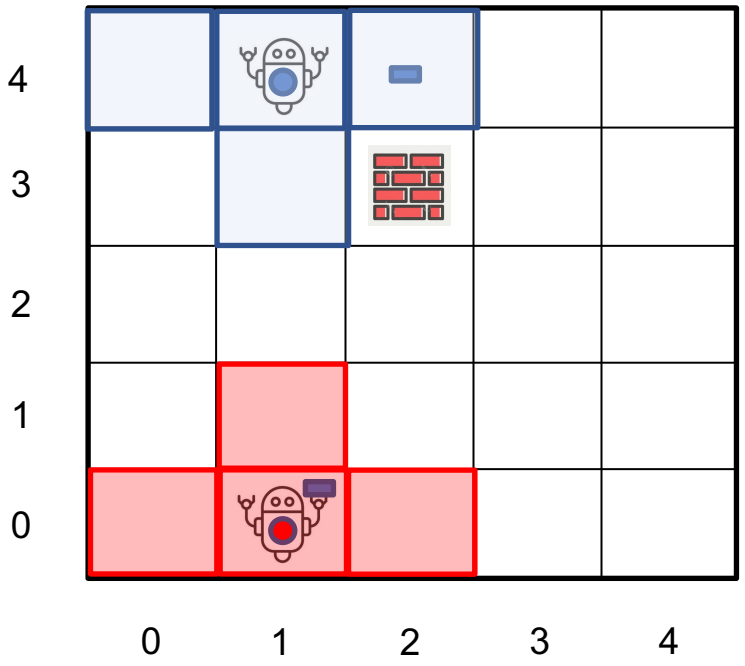
what about t=2 ?

t=2: In(Ag_2,2,2) v In(Ag_2,3,2)



After we had choose beween S1 and S2, should we update beliefs for t=3 and t=2, respectivly?

Partial view of the environment



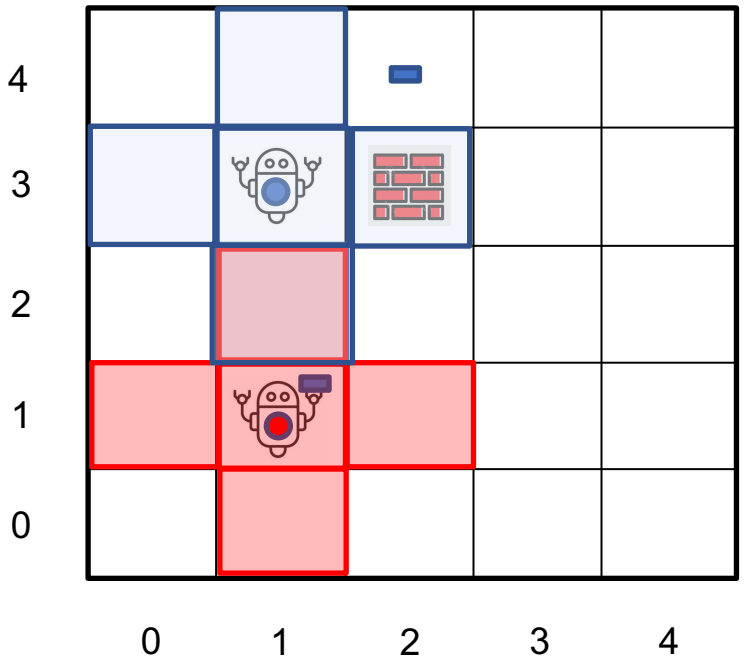
Agent 2

Time	Object	X	Y
1	Ag_2	1	4
1	Pack_1	2	4

Agent 1

Time	Object	X	Y
1	Ag_1	1	0

Partial view of the environment



Agent 2

Time	Object	X	Y
1	Ag_2	1	4
1	Pack_1	2	4
2	Ag_2	1	3
2	Obst_1	2	3

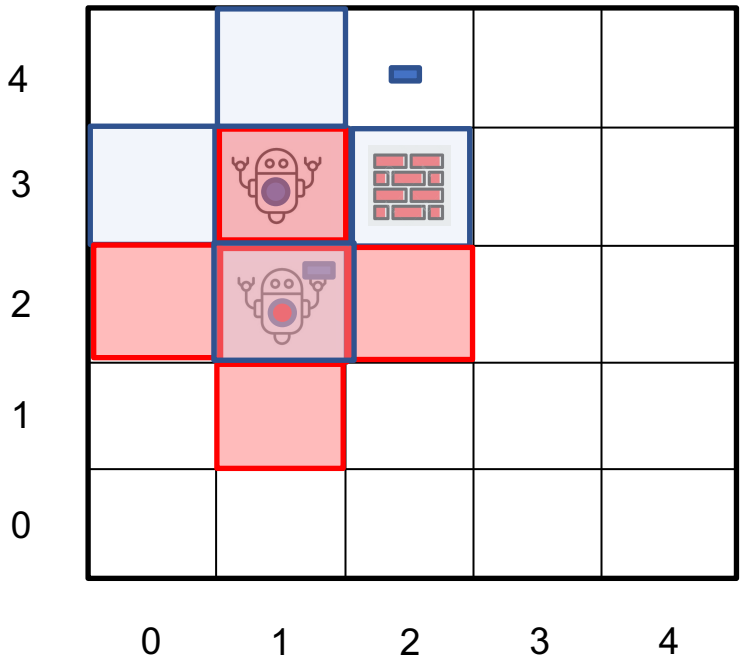
Agent 1

Time	Object	X	Y
1	Ag_1	1	0
2	Ag_1	1	1

At time t=2, is Pack_1 in (2,4)?

- YES ? NO? MAYBE? LIKELY?

Partial view of the environment



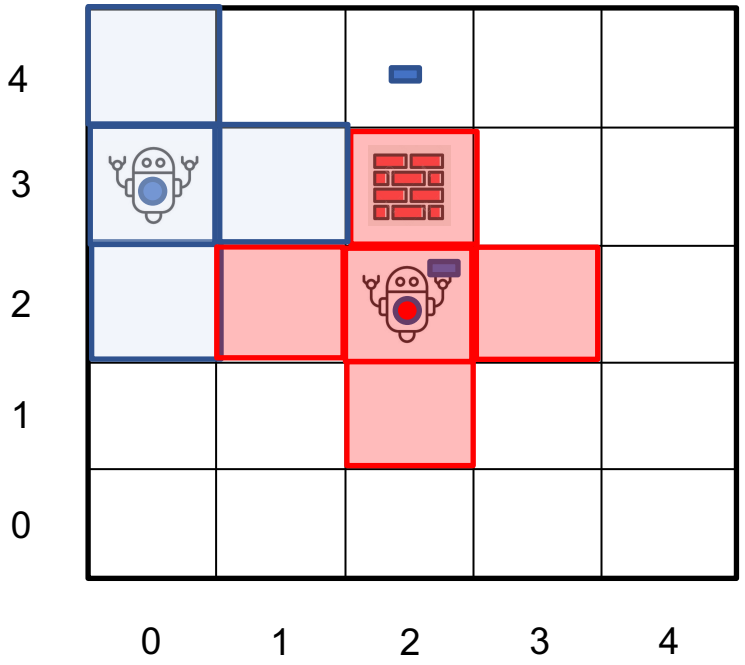
Agent 2

Time	Object	X	Y
1	Ag_2	1	4
1	Pack_1	2	4
2	Obst_1	2	3
2	Ag_2	1	3
3	Obst_1	2	3
3	Ag_2	1	3
3	Ag_1	1	2

Agent 1

Time	Object	X	Y
1	Ag_1	1	0
2	Ag_1	1	1
3	Ag_1	1	2
3	Ag_2	1	3

Partial view of the environment



Agent 2

Time	Object	X	Y
1	Ag_2	1	4
1	Pack_1	2	4
2	Obst_1	2	3
2	Ag_2	1	3
3	Obst_1	2	3
3	Ag_2	1	3
3	Ag_1	1	2
4	Ag_2	0	3

Agent 1

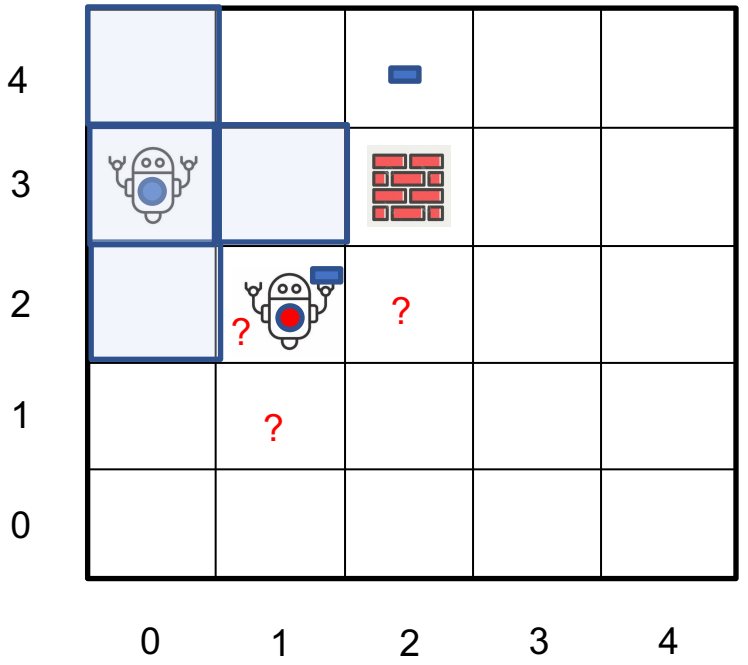
Time	Object	X	Y
1	Ag_1	1	0
2	Ag_1	1	1
3	Ag_1	1	2
3	Ag_2	1	3
4	Ag_1	2	2
4	Obst_1	2	3

They are not anymore there

As in the previous case

Agent 2

Time	Object	X	Y
1	Ag_2	1	4
1	Pack_1	2	4
2	Obst_1	2	3
2	Ag_2	1	3
3	Obst_1	2	3
3	Ag_2	1	3
3	Ag_1	1	2
4	Ag_2	0	3

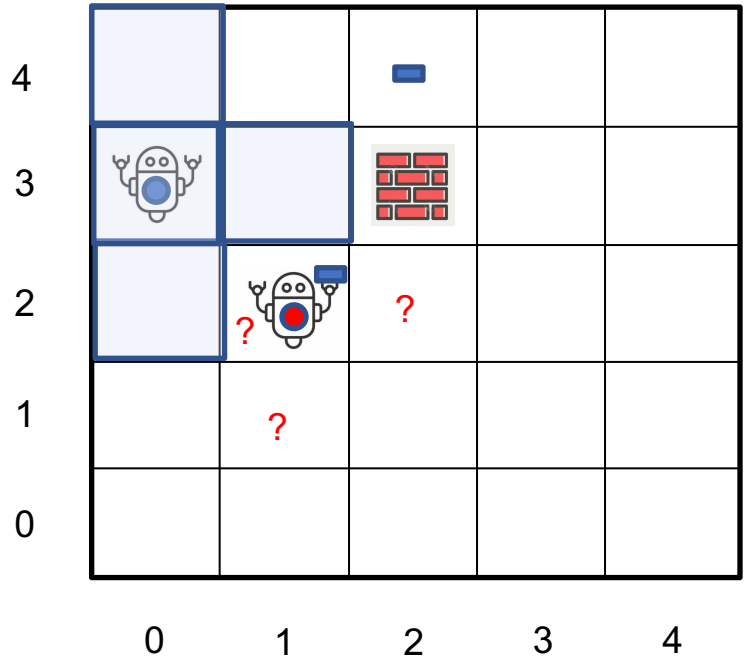


→ `t=4: In(Ag_1,1,2) v In(Ag_1,1,1) v In(Ag_1,2,2)`

Beliefs models

- Several beliefs models can be used
  - No memory: only beliefs based on current data
  - With memory: beliefs based on current data and keeping true not updated beliefs
  - With uncertainty
    - “the probability pack_1 that I saw long time ago is still in position (x,y) is very low”
    - ”the probability obst_1 is in position (x,y) is 1” (obst_1 is a wall a nobody can move it)
    - “I saw Ag_1 going in the direction of Pack_1, the probability Pack_1 is in position (x,y) is very low”
    - “Ag_1 was in position (x,y) and it was moving, the probability it is still there is very low”

In our example



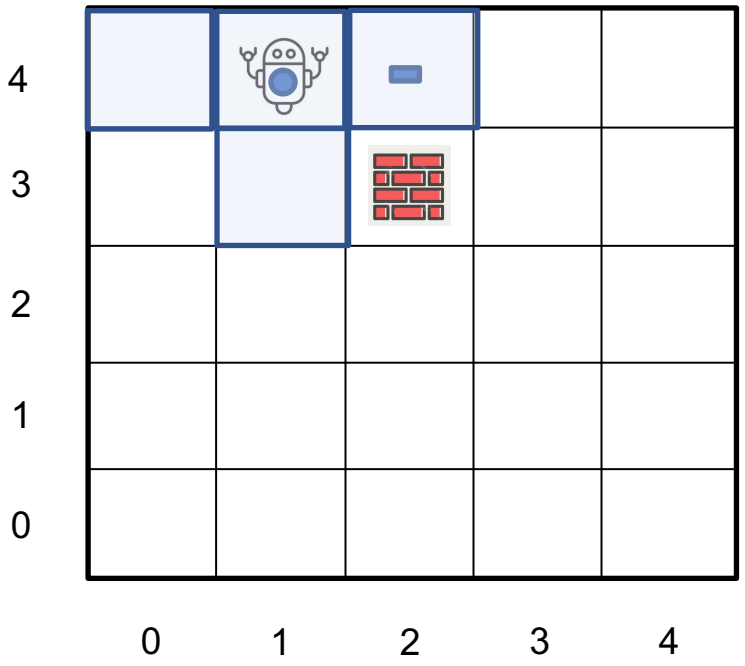
Agent 2

Time	Object	X	Y
1	Ag_2	1	4
1	Pack_1	2	4
2	Obst_1	2	3
2	Ag_2	1	3
3	Obst_1	2	3
3	Ag_2	1	3
3	Ag_1	1	2
4	Ag_2	0	3

→ `t=4: In(Ag_1,1,2) v In(Ag_1,1,1) v In(Ag_1,2,2)`

Time	Object	X	Y	Probability
4	Ag_2	0	3	1
4	Ag_1	1	2	0.33
4	Ag_1	1	1	0.33
4	Ag_1	2	2	0.33

Partial view of the environment



Agent 2

Time	Object	X	Y
1	Ag_2	1	4
1	Pack_1	2	4

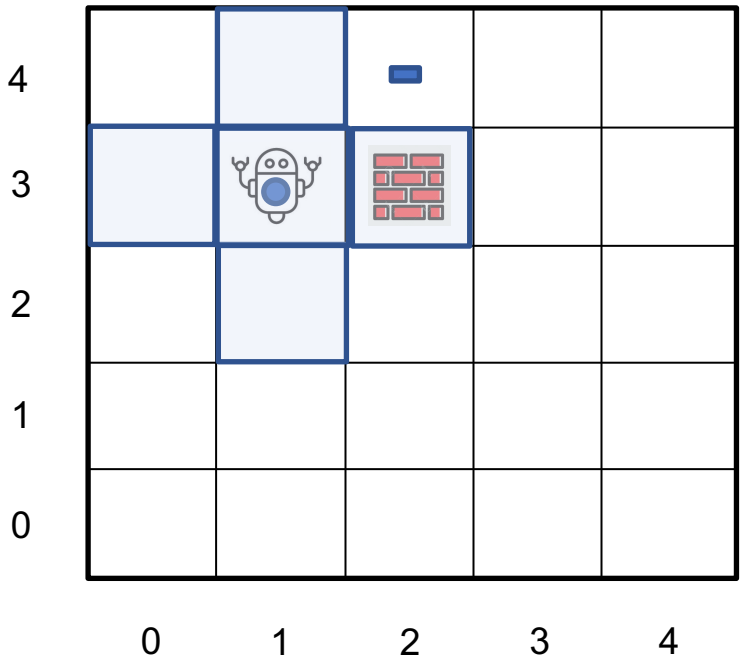
P(Pack_1@2,4) = 0.9

Pack_1 is at (2,4) with confidence **P(Pack_1@2,4) = 0.9`**

- Why 0.9?
- Agent 2 saw it (t=1)
- No info suggests it was picked up yet

it's **very likely** it is there

Partial view of the environment



Agent 2

Time	Object	X	Y
1	Ag_2	1	4
1	Pack_1	2	4
2	Ag_2	1	3
2	Obst_1	2	3

P(Pack_1@2,4) = 0.9

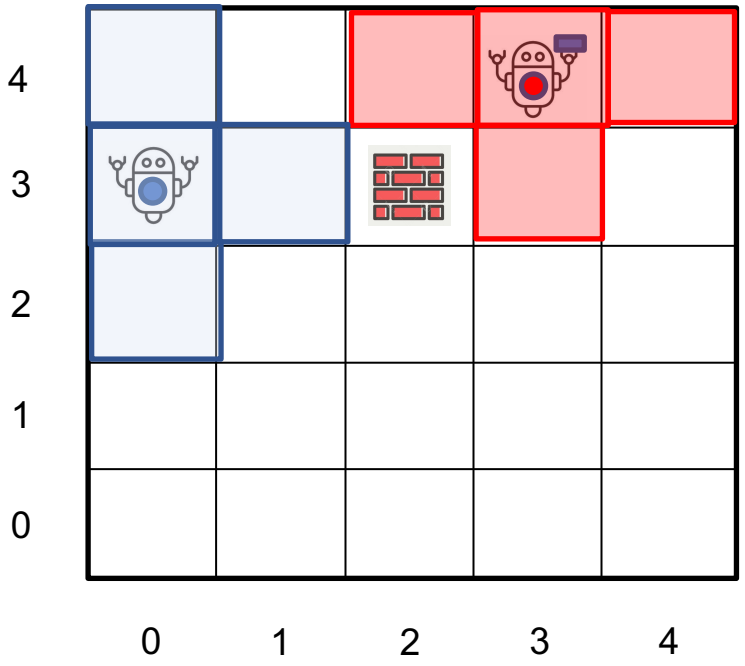
P(Pack_1@2,3 | not seen at t=2) = 0.5

Pack_1 is **not in Agent_2 view range** and it knows that **other agents may have picked it up**

P(P1@2,4 | not seen at t=2) = 0.5

- Partial observability
- Time elapsed since last observation
- Presence of other agents

Partial view of the environment



Agent 2

Time	Object	X	Y
1	Ag_2	1	4
1	Pack_1	2	4
2	Obst_1	2	3
2	Ag_2	1	3
3	Obst_1	2	3

Agent 1 enters view of (2,3)

Agent broadcasts:

¬In(P1,2,3) (Package not found at 2,3)

Agent A2 revises its belief:

- Previous: P(P1@2,4) = 0.5
- New info: A2 did not see Pack_1
- A2 trusts A1's sensor 80% of the time
- Use **Bayesian-style belief revision**

Bayesian-style belief revision

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)}$$

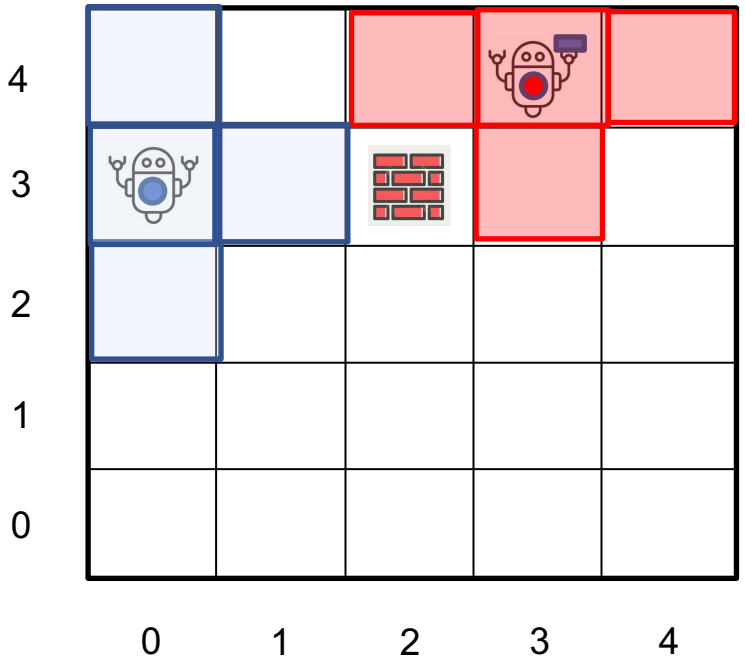
$$P(P1@2,4|\ \neg Seen) = P(\neg Seen\ |\ P1@2,4)\ x\ P(P1@2,4)/\ P(\neg Seen)$$

Assume:

- If P1 is at 2,4 → A1 has 80% chance of seeing it → P(¬Seen | P1@2,4) = 0.2
- If P1 is not at 2,4 → P(¬Seen | ¬P1@2,4) = 1

$$P(P1@2,4|\ \neg Seen) = (0.5x0.2)/[(0.5x0.2)+(0.5x1)] = 0.167$$

Partial view of the environment



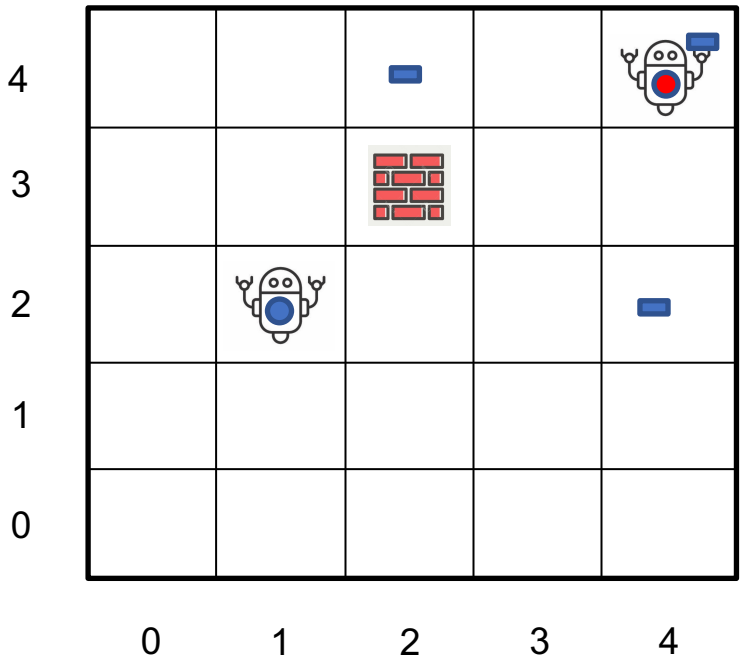
Agent 2

Time	Object	X	Y	P
1	Ag_2	1	4	
1	Pack_2	2	4	**0.9**
2	Obst_1	2	3	
2	Ag_2	1	3	
2	Pack_2	2	4	**0.5**
3	Obst_1	2	3	**1**
3	Ag_2	1	3	
2	Pack_2	2	4	**0.167**

Agent 2 knows that walls cannot be moved away

Since **P(P1@2,4) < threshold (e.g., 0.3)**, A2 **does not generate the option** to pick P1 up and **searches for another package instead**

Partial view of the environment



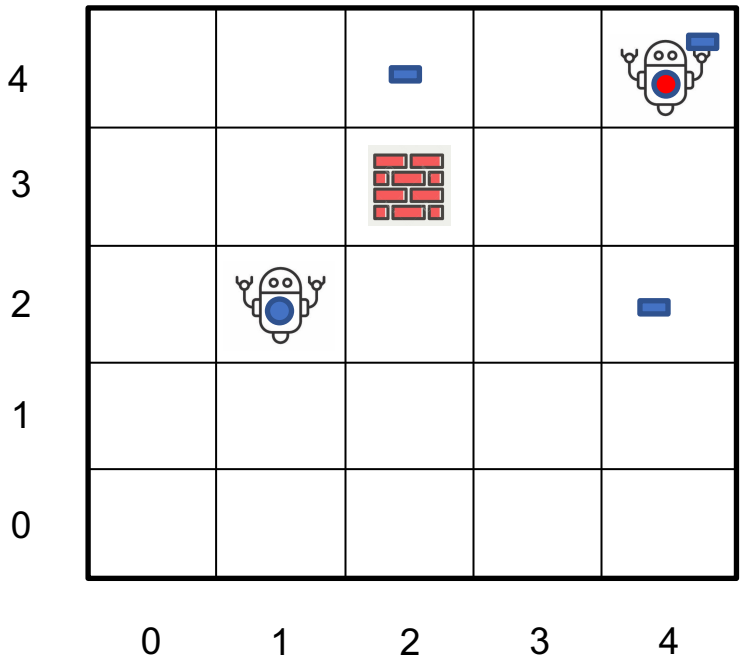
Agent 2

Time	Object	X	Y
1	Ag_2	1	2
1	Pack_1	2	4
1	Pack_2	4	2
1	Ag_1	4	4

Distance Ag_1, Pack_1, Pack_2 = 2

Distance Ag_2, Pack_1, Pack_2 = 3

More on Belief estimation



Agent 2

Time	Object	X	Y	P
1	Ag_2	1	2	
1	Pack_1	2	4	?
1	Pack_2	4	2	?
1	Ag_1	4	4	

How does **Ag_2** estimate the probability that **Pack_1 is still at (2,4)** and **Pack_2 is still at (4,2)** at a future time, given that **Ag_1** might have moved toward them in the meantime?

Belief Estimation

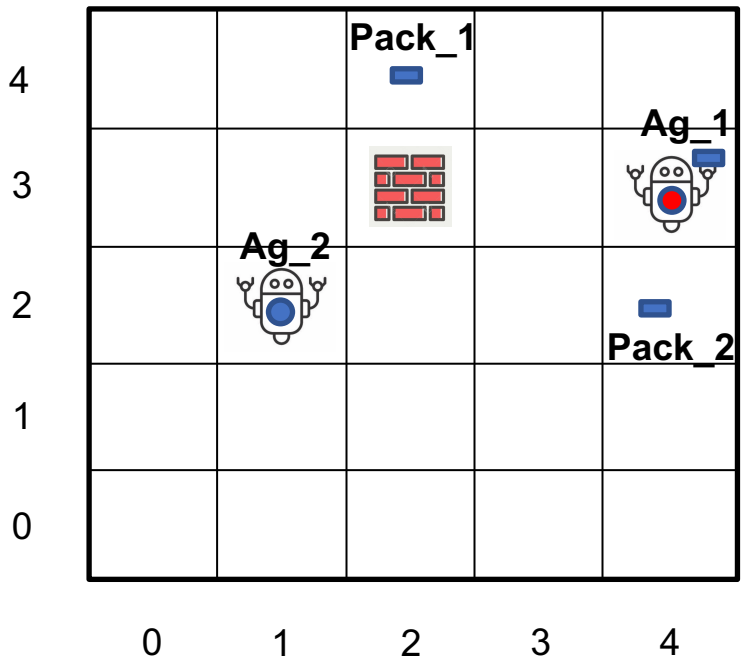
- **Belief estimation problem under uncertainty**, based on:
  - Agent **positions**
  - **Time** since last observation (t=1)
  - **Distance** to targets
  - Risk that another agent has already taken the package

Manhattan Distances

Package	Pos	Ag_1	Ag_2
Pack_1	(2,4)	2	3
Pack_2	(4,2)	2	3

- **Decay function for uncertainty**:
  - $D(d) = e^{-(\lambda \cdot d)}$
  - where *d* is distance from A2, and λ is a decay constant
- **Risk function for A1 stealing the package:**
  - $R = e^{-(\lambda \cdot dA1)}$
  - the closer A1 is, the **higher the risk** of pickup
- Let's pick λ = 0.3
  - A2's distance: 3 → $D = e^{-(0.3 \cdot 3)} \approx 0.406$
  - A1's distance: 2 → $P = 1 - e^{-(0.3 \cdot 2)} \approx 1 - 0.549 = 0.451$
  - Final belief (probability P1 is still there):
    - **P(P1 still there)** = D x P = 0.406 x 0.451 ≈ 0.183
    - P(P2 still there) = P(P1 still there) = 0.183
- **A2 has no strong preference** between the two

Now, consider …



Manhattan Distances

Package	Pos	Ag_1	Ag_2
Pack_1	(2,4)	3	3
Pack_2	(4,2)	1	3

- A1 is **closer** to **P2**
- Both agents are equally distant from **P1**

A2's final belief (probability P1 is still there):

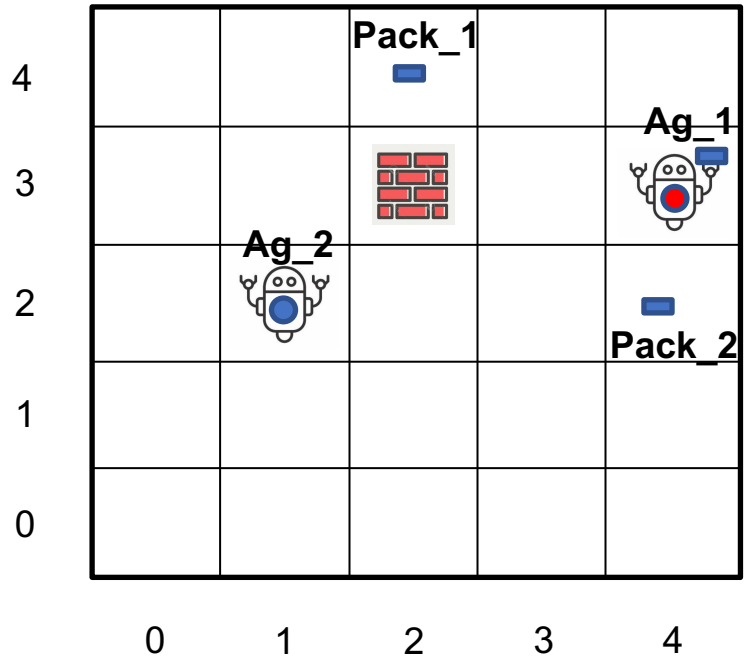
- **P(P2 still there)** = D x (1-R) = 0.406 x (1-0.741) ≈ **0.105**
- **P(P1 still there)** = 0.406 x (1-0.549)= **0.183**

A2 should prioritize going for P1, not P2

… and now

Package	$	Pos	Ag_1	Ag_2
Pack_1	5	(2,4)	3	3
Pack_2	2	(4,2)	1	3

P(P2 still there) = 0.105
P(P1 still there) = 0,183



Expected Value (EV)= P(available) x Reward

EV(P1)=0.183x5=0.915
EV(P2)=0.105x2=0.21

A2 should go for P1, because:
- Even though **it's equally distant**, and **A1 is closer to P2**
- **P1 offers a much better expected return**

But what about A1? Does it not should go for P1 as well?

- Let's define the probability that **A1 goes for a given package** as:

$$P(A1 \rightarrow P_i) = \frac{EV(P_i)}{\sum_j EV(P_j)}$$

EV(P1) = 5 ; EV(P2)=2 ; Total =7

P(A1->P1) = 0.714
P(A1->P2) = 0.286

A2's beliefs $$P(\text{available}) = D(d_{A2}) \cdot (1 - P(A1 \rightarrow P_i) \cdot e^{-\cdot d_{A1}})$$

P(P1) = 0.406 x(1-0.714x 0.406) = 0.288
P(P2) = … = 0.32

→

EV(P1)=0.288x5=1.445
EV(P2)=0.32x2=0.64

Even *after* considering that **A1 is likely to prioritize P1**, the **expected value of going for P1 is still higher** for A2. Now A2's **more informed, realistic belief update** that considers **A1's rationality**.

Beliefs and Introspective abilities

- What about beliefs concerning my intentions, desires, plans, actions? (Introspection)
  - If “I intend G, do I believe I intend to achieve G?”
  - Necessary to reasoning about intentions
    - If there is the opportunity to `pick_up(Pack_2)`
    - What about my current intensions? `B:{Intend(pick_up(Pack_1))}`
    - Are `Intend(pick_up(Pack_1))` and `Intend(pick_up(Pack_2))` consistent?
  - Not easy to implement
  - Synchronization between Beliefs – Intentions – Plans
  - Easy to get into self-contradictory reasoning

Beliefs about other agents' mental states

- Beliefs about other agents' beliefs
  - I believe you believe - `B:{believe(Ag_2,In(Pack_1,2,4))}`
  - Important for coordination, negotiation, and competition
  - We will se more on agents' communication and the speech act theory
- Beliefs about other agents' Intentions
  - I believe you Intend - `B:{intend(Ag_2,pick_up(Pack_1))}`
  - Reasoning about the others' behaviours (coordination, negotiation, and competition)
  - Prediction of others' actions – need to explore possible plans
- Beliefs about other agents' plans
  - I believe you have the plan of `B:{plan(Ag_2,[move(UP),move(RIGHT),pick_up(Pack_1)]}`
  - Usually related to intentions but not always intensions are known
    - I know it will follow that path, but I don't know why
  - Prediction of others' actions

For the project

- We do not use logical deduction

  B=$\{\alpha, \alpha \vdash \beta\}$ means also that $\beta \in B$

- We only use facts (true/false) and environmental contraints (integrety constraint like a DB)

```
fact:       t:   In(Ag_1,1,2)
constraint: t:   In(Ag_1,1,2)
            t+1: In(Ag_1,1,2) v In(Ag_1,1,1) v In(Ag_1,2,2)
```

Implementing the BDI Control Loop

Autonomous Software Agents

A.A. 2025-2026

Prof. Paolo Giorgini



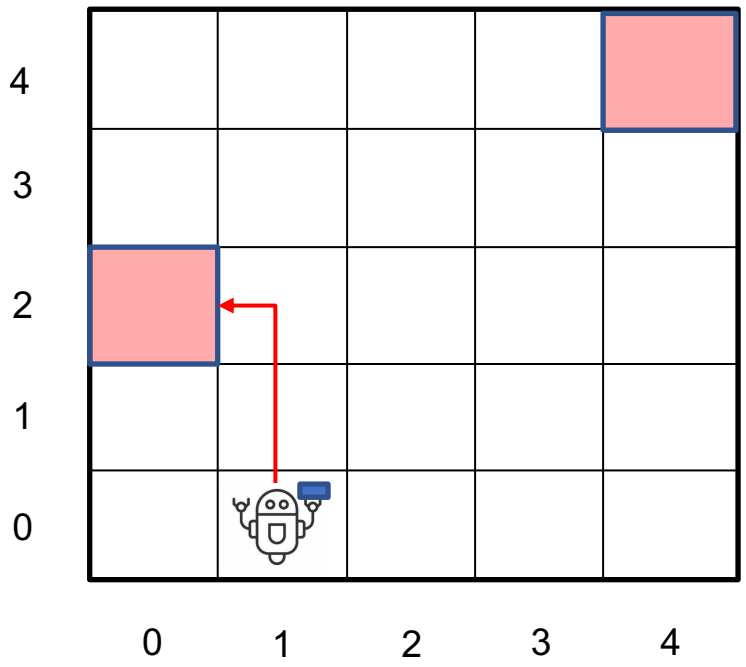
UNIVERSITY OF TRENTO - Italy

Department of Information and Communication Technology

BDI control loop

- Given a set of beliefs B
  1. Decide about possible intentions to adopt
  2. Select new intentions to adopt
  3. Revise the Intention set I
  4. Revise and/or select new plans P for I
  5. Execute plans

```
Agent Control Loop Version 7
1.
2.   B := B_0;
3.   I := I_0;
4.   while true do
5.        get next percept ρ;
6.        B := brf(B, ρ);
7.        D := options(B, I);
8.        I := filter(B, D, I);
9.        π := plan(B, I);
10.       while not (empty(π)
                     or succeeded(I, B)
                     or impossible(I, B)) do
11.            α := hd(π);
12.            execute(α);
13.            π := tail(π);
14.            get next percept ρ;
15.            B := brf(B, ρ);
16.            if reconsider(I, B) then
17.                 D := options(B, I);
18.                 I := filter(B, D, I);
19.            end-if
20.            if not sound(π, I, B) then
21.                 π := plan(B, I)
22.            end-if
23.       end-while
24.  end-while
```




```
-- t=0
B = {in(1,0),carry(pack_1),del_zone(0,2), del_zone(4,4)}
I = {}
P = {}
Do: Null
```

Intention Rules: if (belief) then Int

```
if (carry(Pck) ^ del_zone(X,Y)) { O += in(Pck,X,Y) }

        → O={in(pack_1,0,2), in(pack_1,4,4)}  # options
```

Selection: S = select(B,I,O)

```
                → S={in(pack_1,0,2)}.     # seleted intentions
```

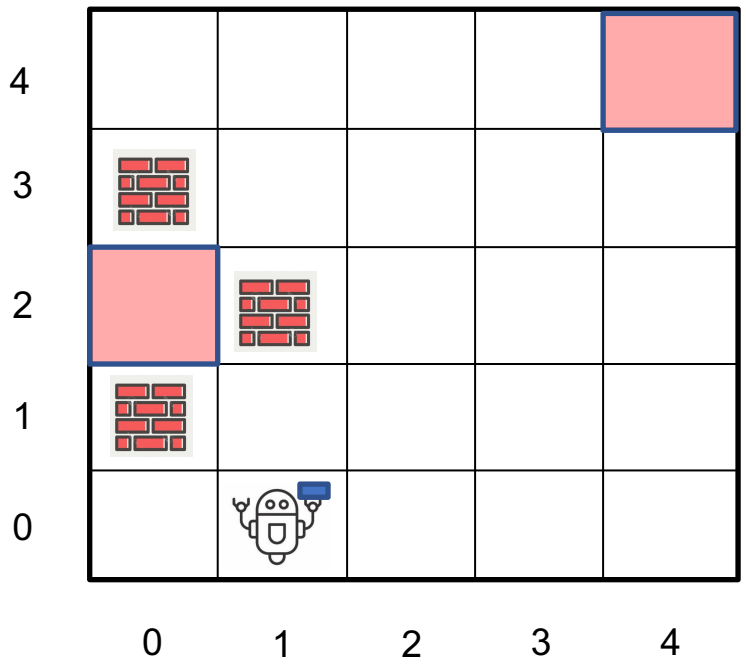
Intentions Revision: I = I_revision(B,I,S)

```
                → I={in(pack_1,0,2)}     # new intentions
```

Plans selection or Plannig: P = planning (B,I,P)

```
        → P = {p1}                              # set of plans for I

        → p1 = {move(UP),move(UP),move(LEFT),put_down(pack_1)}
```




```
-- t=1
B = {in(1,0),carry(pack_1),del_zone(0,2), del_zone(4,4)}
I = {in(pack_1,0,2)}
P = {{move(UP),move(LEFT),put_down(pack_1)}}
Do: move(UP)
```

Options generation:

```
  if (carry(Pck) ^ del_zone(X,Y)) { O += in(Pck,X,Y) }

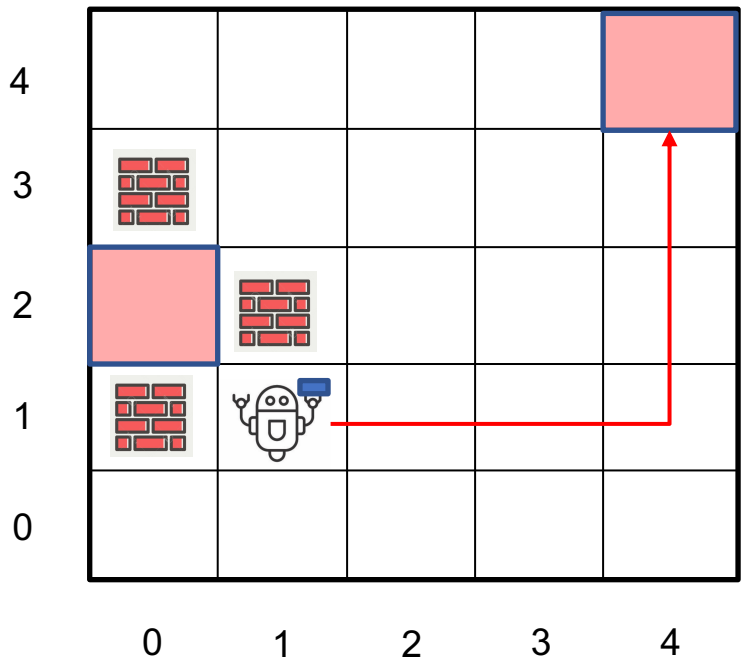
  # again -> in(pack_1,0,2) and in(pack_1,4,4)
  # but nothing changed from t=0 so it will be skipped
```

```
-- t=2
B = {in(1,1),carry(pack_1),del_zone(0,2), del_zone(4,4)
     in(block_1,0,1), in(block_2,1,2), in(block_3,0,3) }
I = {in(pack_1,0,2)}
P = {{move(LEFT),put_down(pack_1)}}
Do: move(UP)
```

Action failure: `move(UP)`

Re-Plannig: `P = planning (B,I,P)`

→ `P = {}` `# no plans for in(pack_1,0,2)`



Intentions Update: I = I_update(B,I, in(pack_1,0,2))

```
# in(pack_1,0,2) is not possible anymore

→ I={}      # in(pack_1,0,2) is dropped
```

Options generation:

```
if (carry(Pck) ^ del_zone(X,Y)) { O += in(Pck,X,Y) }

# again -> in(pack_1,0,2) and in(pack_1,4,4)
# in this case in(pack_1,0,2) should not be considered
# and only in(pack_1,4,4) will be added

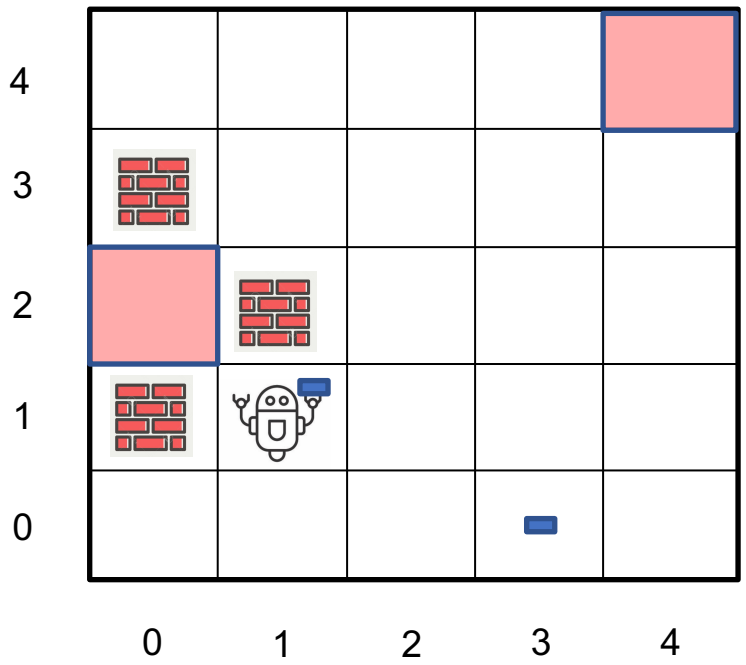
      → O={in(pack_1,4,4)}  # options
```

Selection: → S={in(pack_1,4,4)}.

Intentions Revision: → I={in(pack_1,4,4)}

Plan selection:

```
→ P = {{move(RIGHT), move(RIGHT), move(RIGHT), move(UP),
        move(UP), move(UP), put_down(pack_1)}}
```




```
-- t=3
B = {in(1,1),carry(pack_1),del_zone(0,2), del_zone(4,4)
     in(block_1,0,1), in(block_2,1,2), in(block_3,0,3) }
I = {in(pack_1,4,4)}
P = {{move(RIGHT), move(RIGHT), move(UP),
      move(UP), move(UP), put_down(pack_1)}}
Do: move(RIGHT)

-- t=4
B = {in(2,1),carry(pack_1),del_zone(0,2), del_zone(4,4)
     in(block_1,0,1), in(block_2,1,2), in(block_3,0,3) }
I = {in(pack_1,4,4)}
P = {{move(RIGHT), move(UP), move(UP), move(UP),
      put_down(pack_1)}}
Do: move(RIGHT)

-- t=5
B = {in(3,1),carry(pack_1),del_zone(0,2), del_zone(4,4)
     in(block_1,0,1), in(block_2,1,2), in(block_3,0,3),
     in(pack_2,3,0) }
I = {in(pack_1,4,4)}
P = {{move(UP), move(UP), move(UP), put_down(pack_1)}}
Do: move(RIGHT)
```

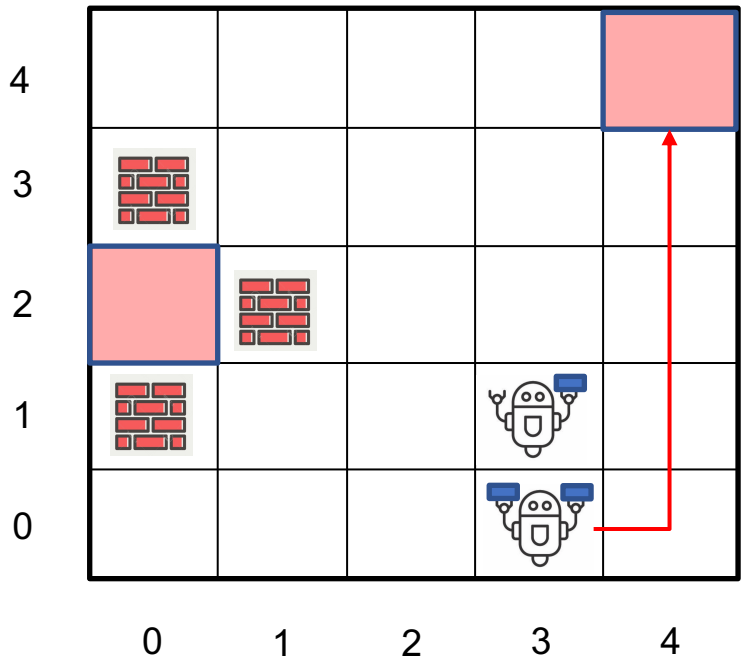
Options generation + selection: → S={pick_up(pack_2)}

Intentions Revision : → I={carry(pack_2), in(pack_1,4,4)}

Re-planning: → I={move(DOWN), pick_up(pack_2)}

```
# with priority on Intentions

# under the hypothesis that Intentions
# cannot run in parallel
```



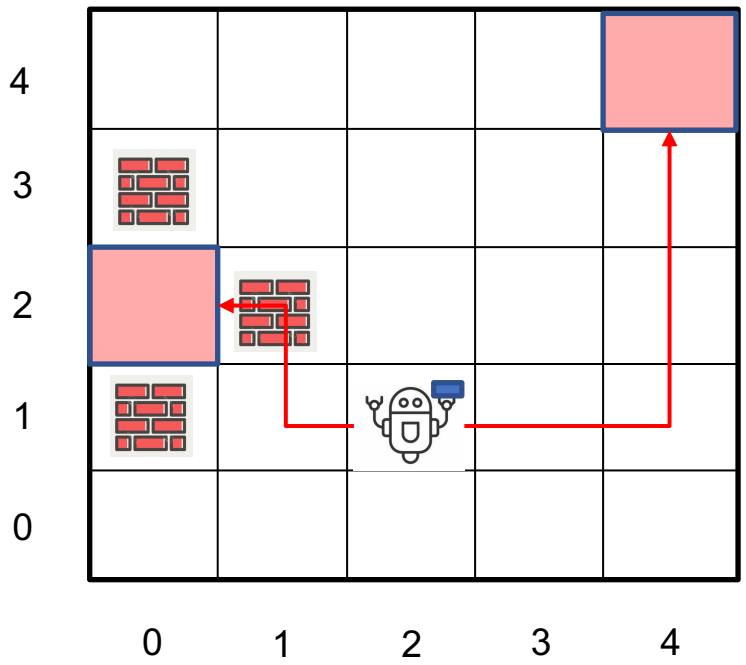

```
-- t=6
B = {in(3,1),carry(pack_1),del_zone(0,2), del_zone(4,4)
     in(block_1,0,1), in(block_2,1,2), in(block_3,0,3),
     in(pack_2,3,0) }
I = {carry(pack_2), in(pack_1,4,4)}
P = {{pick_up(pack_2)}}
Do: move(DOWN)

-- t=7
B = {in(3,0),carry(pack_1),del_zone(0,2), del_zone(4,4)
     in(block_1,0,1), in(block_2,1,2), in(block_3,0,3),
     in(pack_2,3,0) }
I = {carry(pack_2), in(pack_1,4,4)}
P = {}
Do: pick_up(pack_2)

-- t=8
B = {in(3,0),carry(pack_1),del_zone(0,2), del_zone(4,4)
     in(block_1,0,1), in(block_2,1,2), in(block_3,0,3),
     in(pack_2,3,0) }
I = {carry(pack_2), in(pack_1,4,4)}
P = {}
Do: NULL

-- t=9
B = {in(3,0),carry(pack_1),del_zone(0,2), del_zone(4,4)
     in(block_1,0,1), in(block_2,1,2), in(block_3,0,3),
     carry(pack_2)}
I = {in(pack_1,4,4)}
P = {{move(UP), move(UP), move(UP), move(UP), put_down(pack_1)}}
Do: move(RIGHT)
```

Re-Planning for the second intention



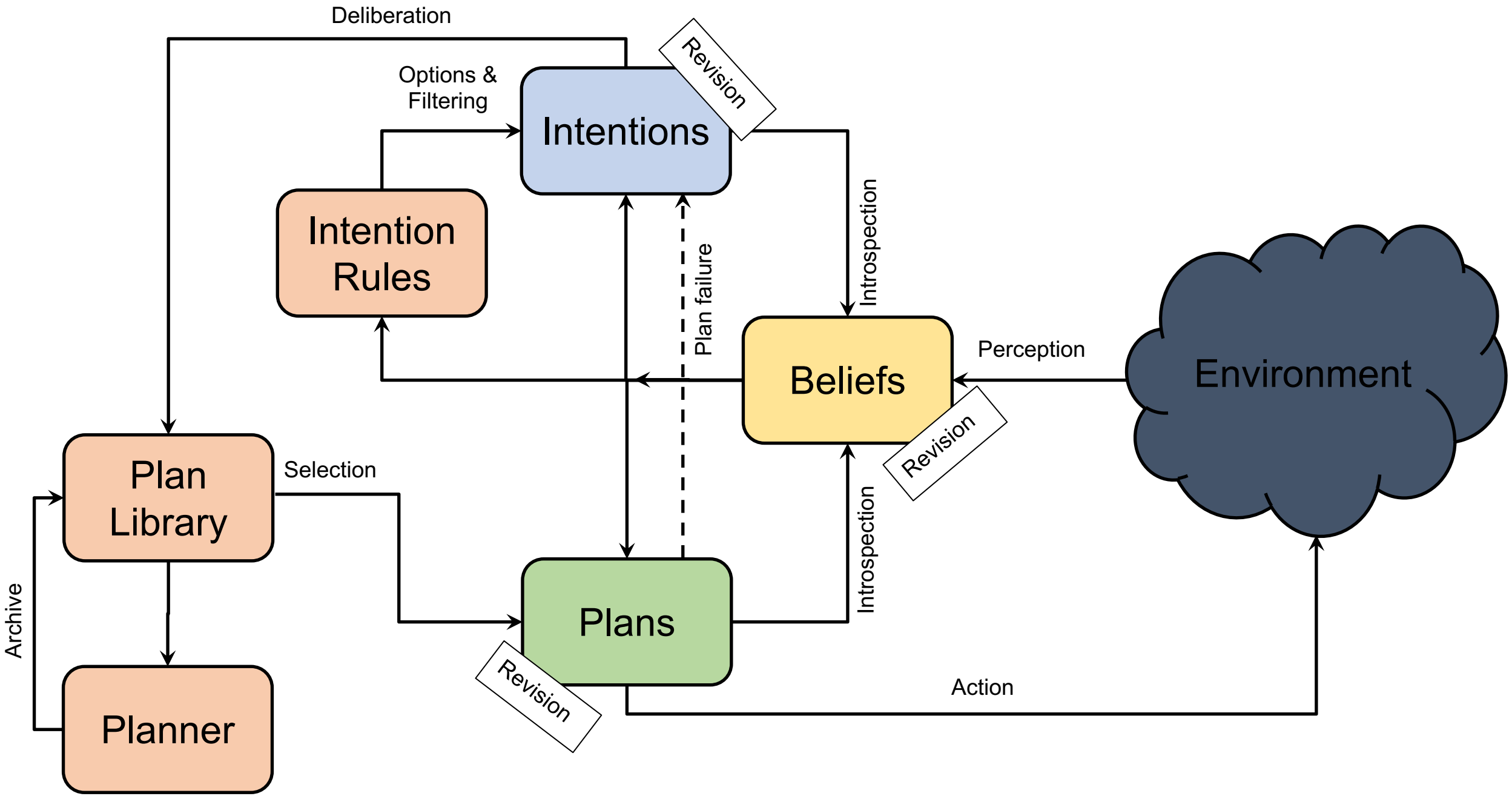

```
-- t=4
B = {in(2,1),carry(pack_1),del_zone(0,2), del_zone(4,4)
     in(block_1,0,1), in(block_3,0,3) }
I = {in(pack_1,4,4)}
P = {{move(RIGHT), move(UP), move(UP), move(UP),
      put_down(pack_1)}}
Do: move(RIGHT)
```

Options generation + selection: → S={in(pack_1,0,2)}

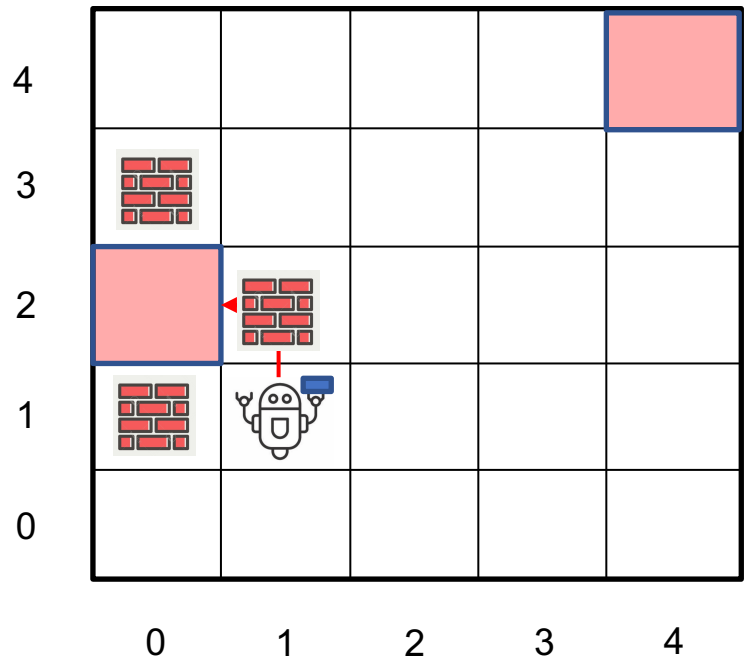
Intentions Revision : → I={in(pack_1,0,2)}

Re-planning: → I={move(LEFT), move(UP), move(LEFT),
put_down(pack_q)}

```
-- t=5
B = {in(2,1),carry(pack_1),del_zone(0,2), del_zone(4,4)
     in(block_1,0,1), in(block_3,0,3) }
I = {in(pack_1,0,2)}
P = {move(UP), move(LEFT), put_down(pack_q)}
Do: move(LEFT)
```


Deliberation
Options &
Filtering
Intentions
Revision
Intention
Rules
Plan failure
Introspection
Beliefs
Perception
Environment
Revision
Plan
Library
Selection
Introspection
Archive
Plans
Revision
Action
Planner

The order of things and timing




```
-- t=0
B = {in(1,1),carry(pack_1),del_zone(0,2), del_zone(4,4)}
I = {}
P = {}
Do: Null
```

Options + Filtering + Int. Revision + Planning

What about if in the meanwhile things in the environment change?

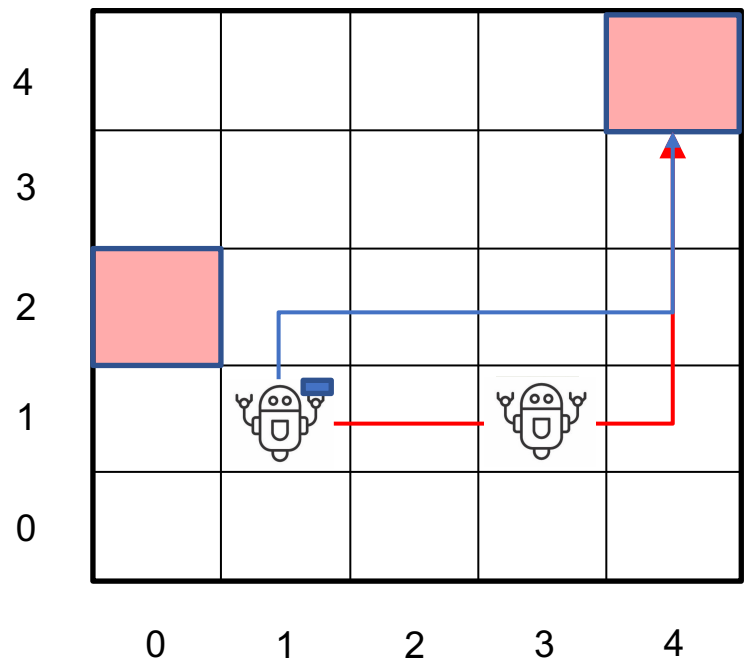
```
in(block_1,0,1), in(block_2,1,2), in(block_3,0,3)
```

```
-- t=1
B = {in(1,1),carry(pack_1),del_zone(0,2), del_zone(4,4)
     in(block_1,0,1), in(block_2,1,2), in(block_3,0,3) }
I = {in(pack_1,0,2)}
P = {{move(LEFT),put_down(pack_1)}}
Do: move(UP)
```

When do we check?

- Much it depends on the application and on the dynamicity of the environment
  - check beliefs at any decision point
    - Option / filtering / deliberation / planning / action
  - check soundness or optimality of the plan before any action
    - Always evaluate the best way to achieve a goal
  - revise intentions before any action
    - This may bring the agent to a schizophrenic behaviour but it could be useful to have agents that are opportunistic

Be careful




```
-- t=0
B = {in(1,1),carry(pack_1),del_zone(0,2), del_zone(4,4)}
I = {in(pack_1,4,4)}
P = {{move(RIGHT), move(RIGHT),move(UP), move(UP), move(UP),
put_down(pack_1)}}
Do: move(RIGHT)
```

The plan is not sound anymore: → Re-planning

```
# since the block will remain in the position (3,1),
# the plan is not sound now and in the future
```

The plan is not sound anymore: → Re-planning ?

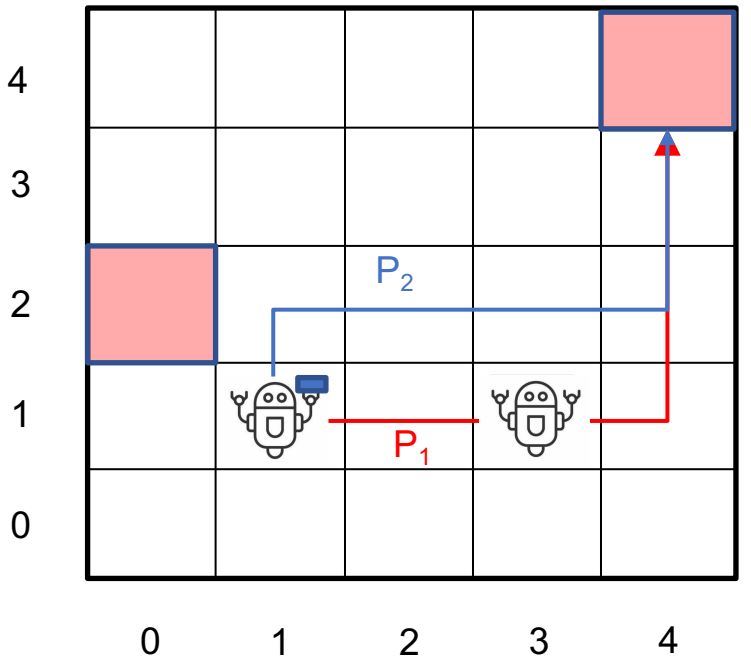
```
# Will the agent remain in the position (3,1)?
# if we replan, it may happen that the new plan will not
# be sound anymore in the next step
```

The agent can distinguish between a block and a moving block

- Different strategies in case of moving blocks

Moving obstacles

- Consider moving obstacles as permanent obstacles
  - No differences in the behaviour
- Elaborating a probability model for the trajectory of the obstacle
  - The new plan will be selected on the base of the probability that the moving obstacle can make the plan not sound at some point (P_1 vs P_2)
- Other strategies
  - Eg. , stay away from other agents or techniques based on game theory



Using the probability model

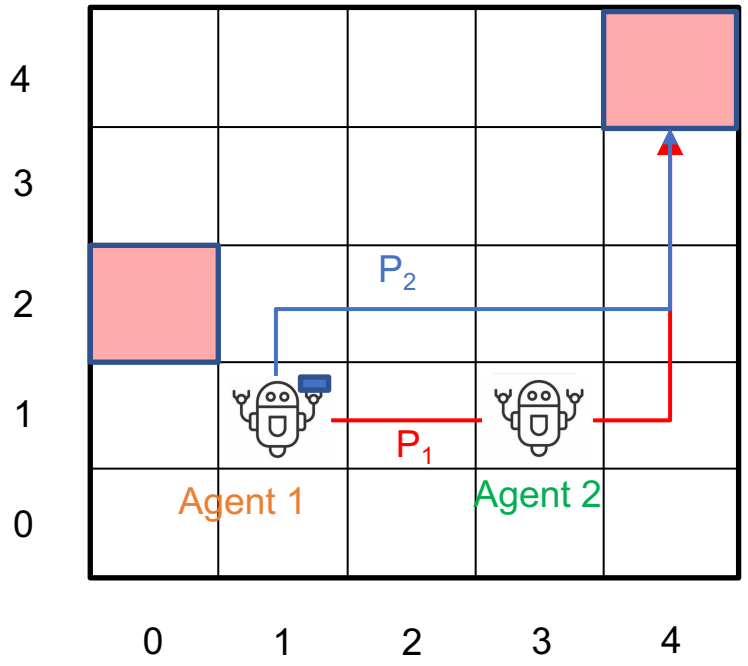
Under the hypothesis of having the same probability that Agent 1 move in any tile or stay where it is

The probability that Agent 2 is an obstacle for Agent 1 executing P_1 is:

- P(move(LEFT)) + P(not move) = 1/5 + 1/5 = 0.4

The probability that Agent 2 is an obstacle for Agent 1 executing P_2 is:

- ….



Intention Revision

Intention Revision

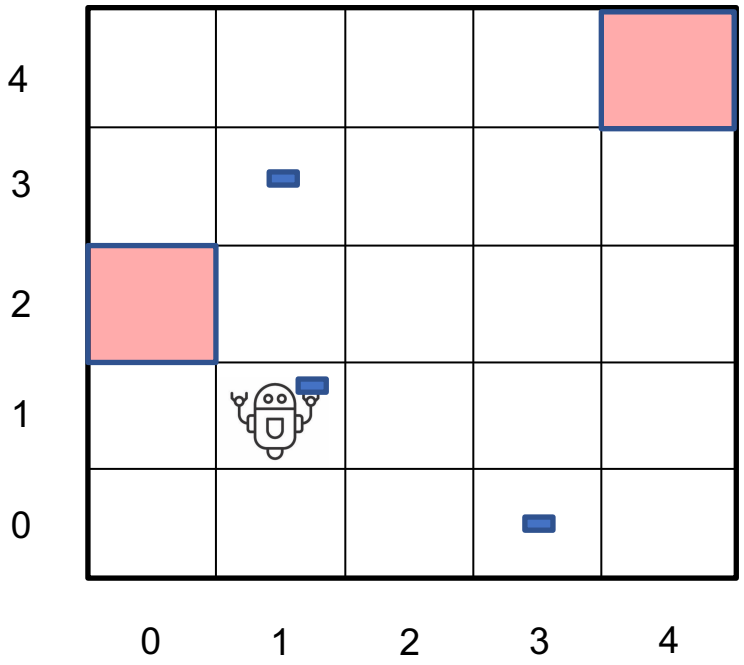
- The process:

  Beliefs Revision -> Options Generation and Selection –> Intentions Revision

- When does an agent need to revise its intentions?
  - Selected options `Opt` and current Intentions `I` are inconsistent
  - Similar to Belief Revision the agent must decide:
    - `Opt` U `I` -> new `I`
    - Note: Intentions can be achieved in parallel or in a sequential order (priority)

    (the agent must schedule the order of the intentions achievement)

Consistency –> re-scheduling




```
-- t=1
B = {in(1,1),carry(pack_1),del_zone(0,2), del_zone(4,4)}
I = {in(pack_1,0,2)}
P = {{move(UP), move(LEFT),put_down(pack_1)}}
Do: Null
```

```
-- t=2
B = {in(1,2),carry(pack_1),del_zone(0,2), del_zone(4,4)
     in(pack_2,3,0), in(pack_3,1,3}
I = {in(pack_1,0,2)}
P = {{move(LEFT), put_down(pack_1)}}
Do: move(UP)
```

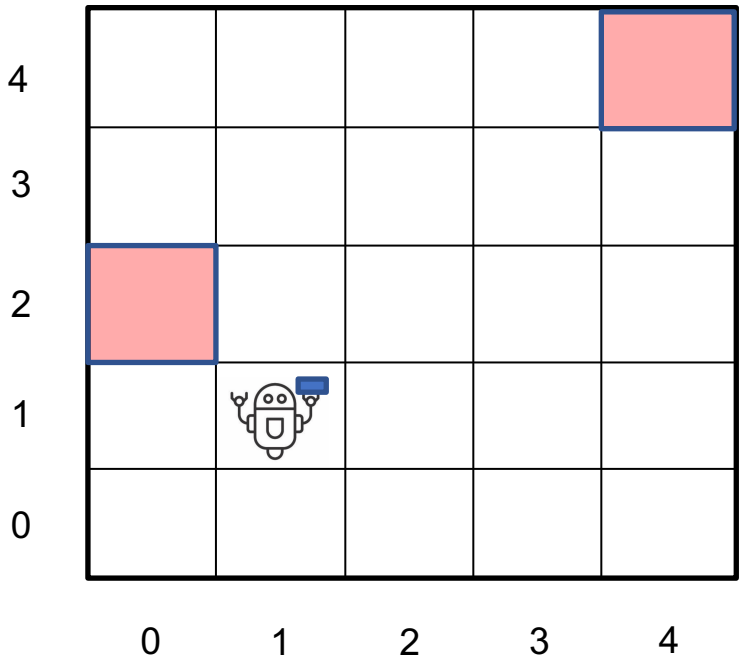
Options generation: → S={pick_up(pack_2), pick_up(pack_3)}

```
   # the agernt can carry max 2 packs
```

```
New I = {in(pack_1,0,2), pick_up(pack_2), pick_up(pack_3)} or
    I = {pick_up(pack_2), in(pack_1,0,2), pick_up(pack_3)} or
    I = {pick_up(pack_3), in(pack_1,0,2), pick_up(pack_2)} or
    I = ….

   # Re-planning is always needed
```

Options and Intentions are inconsistent




```
-- t=1
B = {in(1,1),carry(pack_1), del_zone(4,4)}
I = {in(pack_1,4,4)}
P = {{move(RIGHT), move(RIGHT), move(RIGHT), move(UP),
          move(UP), move(UP), put_down(pack_1)}}
Do: Null

-- t=2
B = {in(2,1),carry(pack_1),del_zone(0,2), del_zone(4,4)}
I = {in(pack_1,4,4)}
P = {{move(RIGHT), move(RIGHT), move(RIGHT), move(UP),
          move(UP), move(UP), put_down(pack_1)}}
Do: move(RIGHT)
```

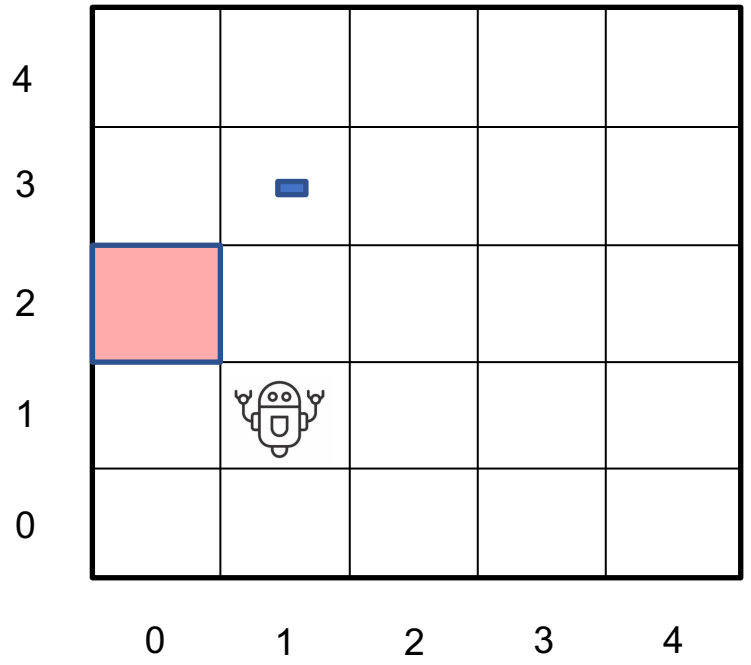
Options generation: → S={in(pack_1,0,2)}

```
   # the option is inconsistent with the current intention
```

New I = {in(pack_1,0,2)} or
I = {in(pack_1,4,5)}

The problem of consequences

If the agent intends α and $\alpha \vdash \beta$




```
-- t=1
B = {in(1,1),in(pack_1,1,3), del_zone(0,2)}
I = {}

# The option is to bring pack_1 in the delivery zone,
# but this means first pick_up the package and then
# delivering it


-- t=2
B = {in(1,1),in(pack_1,1,3), del_zone(0,2)}
I = {carry(pack_1), in(pack_1,1,3)}

# but what about the agent drops the intention to delivery
# pack_1. Does It keep the intention to pick up the package?

# like for beliefs the agent should distinguish between
derived and not derived intentions
```